

FAQ: MongoDB Fundamentals

This document answers some common questions about MongoDB.

What platforms does MongoDB support? [↗](#)

For the list of supported platforms, see [Platform Support](#).

Is MongoDB offered as a hosted service?

Yes. MongoDB Atlas is a cloud-hosted database-as-a-service. For more information, please visit [MongoDB Atlas](#).

How does a collection differ from a table?

Instead of tables, a MongoDB database stores its data in collections. A collection holds one or more BSON documents. Documents are analogous to records or rows in a relational database table. Each document has one or more fields; fields are similar to the columns in a relational database table.

TIP

See also:

- [SQL to MongoDB Mapping Chart](#)
- [Introduction to MongoDB](#)

How do I create a database and a collection?

NOTE

You can enter the commands referenced in this FAQ by using the `MongoDB Shell`. The MongoDB Shell is an interactive JavaScript interface to MongoDB. You can use the MongoDB Shell to query and update data as well as perform administrative operations.

If a database does not exist, MongoDB creates the database when you first store data for that database.

If a collection does not exist, MongoDB creates the collection when you first store data for that collection.

As such, you can switch to a non-existent database (use `<dbname>`) and perform the following operation:

```
use myNewDB;  
db.myNewCollection1.insertOne( { x: 1 } );  
db.myNewCollection2.createIndex( { a: 1 } );
```



- The `db.collection.insertOne()` method creates the collection `myNewCollection1` if it does not already exist.

- The `db.collection.createIndex()` method creates the index and the collection `myNewCollection2` if it does not already exist.
- If the `myNewDb` database did not exist, either the `db.collection.createIndex()` method or `db.collection.insertOne()` method would have created the `myNewDb` database automatically.

You can also create a collection explicitly using `db.createCollection()` method if you want to specify specific options, such as maximum size or document validation rules:

```
use myNewDB;  
db.createCollection("myNewCollection1");
```



How do I define or alter the collection schema?

You do not need to specify a schema for a collection in MongoDB. Although it is common for the documents in a collection to have a largely homogeneous structure, it is not a requirement; i.e. documents in a single collection do not need to have the same set of fields. The data type for a field can differ across documents in a collection as well.

To change the structure of the documents in a collection, update the documents to the new structure. For instance, add new fields, remove existing ones, or update the value of a field to a new type.

NOTE

You can enforce document validation rules for a collection during update and insert operations.

Some collection properties, such as specifying a maximum size, can be specified during the explicit creation of a collection and be modified. See `db.createCollection()` and `collMod`. If you are not specifying these properties, you do not need to explicitly create the collection since MongoDB creates new collections when you first store data for the collections.

Does MongoDB support SQL?

Not directly, no. However, MongoDB does support a rich query language of its own. For examples on using MongoDB's query language, see MongoDB CRUD Operations.

You can also use the MongoDB Connector for BI to query MongoDB collections with SQL.

If you are considering migrating your SQL application to MongoDB, download the MongoDB Application Modernization Guide for a best practices migration guide, reference schema, and other helpful resources.

TIP

See also:

SQL to MongoDB Mapping Chart

Does MongoDB support transactions?

Because a single document can contain related data that would otherwise be modeled across separate parent-child tables in a relational schema, MongoDB's atomic single-document operations already provide transaction semantics that meet the data integrity needs of the majority of applications. One or more fields may be written in a single operation, including updates to multiple sub-documents and elements of an array. The guarantees provided by MongoDB ensure complete isolation as a document is updated; any errors cause the operation to roll back so that clients receive a consistent view of the document.

For situations that require atomicity of reads and writes to multiple documents (in a single or multiple collections), MongoDB supports distributed transactions, including transactions on replica sets and sharded clusters.

For more information, see [transactions](#).

IMPORTANT

In most cases, a distributed transaction incurs a greater performance cost over single document writes, and the availability of distributed transactions should not be a replacement for effective schema design. For many scenarios, the denormalized data model (embedded documents and arrays) will continue to be optimal for your data and use cases. That is, for many scenarios, modeling your data appropriately will minimize the need for distributed transactions.

For additional transactions usage considerations (such as runtime limit and oplog size limit), see also [Production Considerations](#).

Does MongoDB handle caching?

Yes. MongoDB keeps most recently used data in RAM. If you have created indexes for your queries and your working data set fits in RAM, MongoDB serves all queries from memory.

MongoDB does not cache the query results in order to return the cached results for identical queries.

For more information on MongoDB and memory use, see [WiredTiger](#) and [Memory Use](#).

How does MongoDB address SQL or Query injection?

BSON

As a client program assembles a query in MongoDB, it builds a BSON object, not a string. Thus traditional SQL injection attacks are not a problem. More details and some nuances are covered below.

MongoDB represents queries as BSON objects. Typically client libraries provide a convenient, injection free, process to build these objects. Consider the following C++ example:

```
BSONObj my_query = BSON( "name" << a_name );  
auto_ptr<DBClientCursor> cursor = c.query("tutorial.persons", my_query);
```



Here, `my_query` then will have a value such as `{ name : "Joe" }`. If `my_query` contained special characters, for example `,`, `:`, and `{`, the query simply wouldn't match any documents. For example, users cannot hijack a query and convert it to a delete.

JavaScript

NOTE

You can disable all server-side execution of JavaScript:

- For a `mongod` instance by passing the `--noscripting` option on the command line or setting `security.javascriptEnabled` to false in the configuration file.
- For a `mongos` instance by passing the `--noscripting` option on the command line or setting `security.javascriptEnabled` to false in the configuration file.

The following MongoDB operations permit you to run arbitrary JavaScript expressions directly on the server:

- `$where`
- `mapReduce`
- `$accumulator`
- `$function`

You must exercise care in these cases to prevent users from submitting malicious JavaScript.

Fortunately, you can express most operations in MongoDB without JavaScript.